

Data Engineer Dev Assignment — Azure Medallion Data Product

Role: Data Engineer

Estimated effort: a focused take-home (quality over completeness)

Submission: GitHub repository — grant access to `dp1-tech-uk`

Share this **single file** (`data-engineer-dev-assignment`) with candidates. It contains:

Section	Purpose
Part A — Data Engineer Dev Assignment	Problem, requirements, deliverables (give to candidates)
Part B — Evaluation	Review rubric (optional to share; used when scoring)

Part A — Data Engineer Dev Assignment

1. Goal

Build a thin **Azure medallion** pipeline that ingests university chapter data from a public API and publishes it as a **data product** others can consume.

You are **not** building a data mesh. You **are** expected to treat the published Gold layer as a deliberate **product**: clear schema, ownership, quality bar, and a contract a consumer could rely on.

2. Business scenario

We need a reusable dataset of university chapters to support analytics and reporting.

Source API (public ArcGIS FeatureServer):

Use	URL
Ingest (use this)	<code>https://services2.arcgis.com/5I7u4SJE1vUr79JC/arcgis/rest/services/UniversityChapters_Public/FeatureServer/0</code>
Query	<code>.../FeatureServer/0/query</code>
Hub page (HTML only — do not scrape as JSON)	<code>https://gis.ducks.org/datasets/du-university-chapters/api</code>

Example query:

```
GET .../FeatureServer/0/query
  ?where=State IN ('CA', 'OR', 'WA')
  &outFields=*
  &returnGeometry=true
  &f=json
```

Layer is point geometry (WGS84). Upstream view already filters `Status = 'ACTIVE'`. Pagination exists (`maxRecordCount` 1000) but is unnecessary for west-coast volume.

Scope — only these states:

State	Code	Expected volume (baseline)
California	CA	Non-zero (e.g. 3 ACTIVE)
Oregon	OR	May be zero
Washington	WA	May be zero

Empty OR/WA is a **source data fact**, not a failed pipeline. Still apply the three-state filter so your data-quality story can distinguish “CA missing when historically present” from “OR/WA empty.”

Minimum attributes to capture and publish:

Source field	Publish as	Notes
ChapterID	chapter_id	Stable business key (e.g. CA-0355)
University_Chapter	chapter_name	Display name
City	city	
State	state	USPS 2-letter; filter CA / OR / WA
geometry.x / geometry.y	longitude / latitude	WGS84

Optional to land but keep out of the consumer contract (or mark technical): OBJECTID → source_object_id.

3. What to build

3.1 Medallion layers (required)

Layer	Purpose	Expectation
Bronze	Raw land — API payload as received (or near-as-received), plus ingest metadata	History by run; no business transforms
Silver	Cleaned, typed, deduped, flattened to chapter grain	Apply required DQ (§3.3): quarantine fails vs warn-and-pass
Gold	Published data product — consumer-facing	Clean + warned rows only (no quarantined rows)

3.2 Technology expectations

Use **Azure-oriented** data engineering patterns and **Spark** (or a clear Spark-shaped transform layer) for Silver/Gold.

Typical choices candidates use:

Concern	Examples
Storage / lake layout	Paths or tables for bronze/ silver/ gold/ and quarantine/ (ADLS-style layout is fine even if files are local)
Compute	PySpark, Databricks, Fabric Spark, Synapse Spark, or local Spark
Ingest	Python (or notebook) calling the FeatureServer REST API

Out of scope for this exercise: IaC (Terraform/Bicep), CI/CD, multi-environment promotion, schedulers, streaming, and production platform landing zones.

Document clearly in the README **how a reviewer runs your solution** (commands, notebooks, or steps). Prefer something reproducible from a clone (fixtures + local run is acceptable if cloud setup is heavy).

Secrets: this public API needs no credentials. Do **not** commit secrets. If you add any config, use ignored local env files only.

3.3 Data quality — quarantine vs warning (required)

Implement **both** behaviours on the Bronze → Silver path. Live API rows may all be clean; **seed at least one synthetic bad row** in a fixture (or inject in Bronze) so reviewers can see each path.

Rule ID	Severity	Condition (fail when...)	What happens
DQ-Q1	Quarantine (hard fail)	longitude / latitude missing, null, non-numeric, or outside valid ranges (lon $\notin$ [-180,180] or lat $\notin$ [-90,90])	Row does not enter Silver/Gold. Write it to a quarantine path/table with reason code INVALID_COORDINATES, plus ingest_run_id and enough raw payload to debug.
DQ-W1	Warning (soft)	city is null, blank, or the literal UNKNOWN (case-insensitive)	Row still publishes to Silver → Gold. Set dq_status = 'WARNING' and append reason MISSING_OR_UNKNOWN_CITY to dq_warnings (array or delimited string). Clean rows use dq_status = 'OK' and empty warnings.

Also required:

- Quarantine and Gold must be **visibly different** (two paths or two tables).
- Log counts per run: rows_in, rows_quarantined, rows_warned, rows_ok.
- Quarantined rows must **never** appear in Gold.
- Document both rules in the Data Product Contract.

Suggested layout:

```
.../bronze/university_chapters/<run_id>/
.../silver/university_chapters/
.../gold/university_chapters/v1/
.../quarantine/university_chapters/<run_id>/ # DQ-Q1 only
```

3.4 Data product contract (required)

Ship a short **Data Product Contract** (markdown or YAML) covering:

1. **Name & owner** — product name, technical owner, consumer use cases
2. **Interface** — table / path / schema (column names, types, grain), including dq_status / dq_warnings
3. **Freshness** — intended SLA (e.g. “refreshed daily by 06:00 UTC”) even if your run is manual
4. **Quality** — DQ-Q1 / DQ-W1 plus batch rules: **do not** require OR/WA row counts > 0; do fail or alert if the whole batch is empty or CA drops to zero unexpectedly
5. **Versioning** — how breaking schema changes would be handled (e.g. v1 path / table suffix)
6. **Classification** — public source data; no PII expected; label accordingly

4. Deliverables

#	Deliverable	Done when
D1	GitHub repo with README	Clone → run steps are clear
D2	Short architecture note + diagram	Medallion + chosen tech visible
D3	Working Bronze → Silver → Gold path	Produces Gold for clean CA data
D4	Spark (or equivalent) transforms	Logic is reviewable in code / notebooks
D5	Data Product Contract	Sections in §3.4 present, including DQ-Q1 / DQ-W1
D6	DQ quarantine + warning	Fixture-driven: one row in quarantine , one warned row in Gold
D7	Tests or automated checks	Assert DQ-Q1 excluded from Gold; DQ-W1 present with warning

Optional stretch: extra DQ rules · partitioned Bronze by `ingest_date` · idempotent Gold MERGE.

5. Explicit non-goals

- Data mesh / federated domain platform work
- Terraform, Bicep, or other IaC
- CI/CD pipelines or multi-environment promotion
- Production schedulers, catalogues, or secret-store wiring
- Perfect SCD2 history (append-by-run in Bronze is enough)

6. Engineering bar

- **No secrets in Git**
- **Idempotent-ish publishes** — re-running should not create unusable Gold chaos (overwrite partition or MERGE by `chapter_id` — state your choice)
- **Fail loudly** — API errors and empty result sets must not silently publish empty Gold as “success” without a clear policy
- **Explain trade-offs** briefly in the README (tooling choices, what you would add in production)
- **Original work** — do not paste production code from other employers’ platforms

7. Submission

1. Push your solution to a **GitHub** repository (private is fine).
2. Grant access to the GitHub user/org **dp1-tech-uk**.
3. Reply with the repository URL when ready.

Reviewers will clone the repo, follow your README, and score against [Part B](#).

Part B — Evaluation

For hiring reviewers. Score each area 0–2. You may share this section with candidates if you want expectations transparent.

Score	Meaning
0	Missing / unclear
1	Partial / works but thin
2	Solid for a take-home

Pass bar: $\geq 10 / 14$ and the Gold path actually runs from the README.

Rubric

Area	What to look for	0–2
Medallion clarity	Bronze/Silver/Gold have distinct jobs; raw not mixed into Gold	
Azure + Spark mindset	Spark (or equivalent) transforms; lake/medallion layout clear	
Correct scope	CA/OR/WA filter; required fields mapped; coords usable; OR/WA empty handled as data fact	
Data product	Contract exists; Gold schema stable and documented	
Quality & failure	DQ-Q1 quarantine + DQ-W1 warn-and-pass both evidenced; bad rows never in Gold	
Engineering hygiene	No secrets in repo; README runnable; sensible structure	
Trade-offs & next steps	Clear about choices and what production would add (ideas only)	

Total: ____ / 14

Fast fail

- Gold does not materialise when CA data is available (OR/WA-only empty is OK)
- No quarantine path, or warned rows treated the same as hard fails (or vice versa)
- Secrets committed
- Single flat dump with no medallion separation
- “Data product” means only a screenshot of a dataframe with no contract
- Repo not shared with **dp1-tech-uk** / README cannot be followed

Praise signals (not required)

- Partitioned Bronze by ingest_date
- MERGE / idempotent Gold
- Fixture tests covering both DQ-Q1 and DQ-W1 offline
- Clean, reviewable code structure and logging of DQ counts